

PREN Delta Robot Controller — Dokumentation

Pascal Hofstetter, Roman Winiger

2026

Table of Contents

Übersicht	1
Team	1
Softwarearchitektur	2
Modulverantwortlichkeiten	2
Kompilierzeit-Konfigurationsflags	2
Hardware	4
Mikrocontroller	4
Pin-Belegung	4
Schrittmotoren	4
Sensoren (Endschalter)	4
Spule (Elektromagnet)	4
Debug- / Reserve-Pins	4
Timer — FTM0-Kanal-Zuordnung	5
Kommunikation	6
UART-Schnittstelle	6
Befehlssatz	6
Terminal-Handler	6
Motorsteuerung	7
Schritimpuls-Erzeugung	7
Überlaufbehandlung	7
Rampenprofile	7
Referenzfahrt	7
Erstellen & Flashen	8
Voraussetzungen	8
Erstellen mit MCUXpresso	8
Erstellen über die Kommandozeile	8
Flashen & Debuggen	8
Debug-Modi	8

Übersicht

Dieses Projekt implementiert die eingebettete Firmware für einen **Delta-Roboter**, der im Rahmen des PREN-Moduls (Praktisches Projekt in Elektronik/Nachrichtentechnik) an der **Hochschule Luzern T&A**, Gruppe 1, 2026 entwickelt wurde.

Die Firmware läuft auf einem **NXP Kinetis K22 (MK22F51212)** Mikrocontroller und ist verantwortlich für:

- Empfang von Bewegungsbefehlen von einem Raspberry Pi über UART
- Ansteuerung von drei linearen Schrittmotoren (Delta-Achsen)
- Steuerung eines Rotationsmotors am Endeffektor
- Aktivierung eines Elektromagneten (Spule) zur Objektmanipulation
- Auslesen von Endschalter-Sensoren für die Referenzfahrt

Team

Name	Rolle
Pascal Hofstetter	Firmware-Entwicklung
Roman Winiger	Firmware-Entwicklung

Softwarearchitektur

Die Firmware ist als Bare-Metal-C (kein RTOS) mit einem modularen Schichtendesign aufgebaut.

source/	
— main.c	Einstiegspunkt, Testsequenzen
— platform.h	Takt-Konfigurationen, Fehlercodes, Debug-Makros
— globals.h	Globaler Motorzustand und Timer-Parameter
— control/	Roboter-Steuerlogik
— control.c/h	Zentraler Befehls-Dispatcher
— motor/	Schrittmotor-Treiber
— sensor/	Endschalter-Sensoren
— coil/	Elektromagnet-Steuerung
— com/	Kommunikation
— uart2.c/h	UART2-Treiber (115200 Baud)
— term.c/h	Terminal-Befehlshandler
— utils/	Hilfsfunktionen
— ftm0.c/h	FTM0-Timer (Schrittpuls-Erzeugung)
— wait.c/h	Verzögerungsfunktionen
— debug.c/h	Debug-Hilfsfunktionen

Modulverantwortlichkeiten

Modul	Verantwortlichkeit
control	Initialisiert alle Subsysteme; leitet empfangene Befehle an Motor-, Spulen- und Sensormodule weiter
motor	Erzeugt präzise Schrittpulse über FTM0-Interrupt-Kanäle; verwaltet Rampenprofile und Überlauf
calculation	Wandelt die gewünschte Roboterposition in einzelne Achsen-Schrittzahlen um (Kinematik)
sensor	Liest den Endschalter-Zustand an PC8, PC9, PC10 für die Referenzfahrt
coil	Schaltet den Elektromagneten an PD0
term	Parst UART-Befehle und leitet sie an registrierte Handler weiter
ftm0	Konfiguriert FTM0-Kanäle und bedient ISRs für die Schritt-Zeitmessung

Kompilierzeit-Konfigurationsflags

Definiert in `globals.h`:

Flag	Wirkung
DEBUG_MODE	Aktiviert Debug-Pin-Überwachung, deaktiviert die Hauptschleife

Flag	Wirkung
TEST_SEQUENCE	Führt beim Start einen automatisierten Bewegungstest aus (erfordert DEBUG_MODE)
RAMP_ACTIVE	Aktiviert die Motoranlauframpe
RAMP_MODE_PS	Rampe über Prescaler-Modulation (4 Stufen)
RAMP_MODE_NSTEP	Rampe über N-Schritt-Profil (Standard: 20 Schritte)
RAMP_MODE_PREMIUM	Lineare Rampe, in der ISR berechnet
ISR_MONITOR	Gibt ISR-Timing auf Reserve-Debug-Pins aus
SIM_SENSORS	Simuliert Sensorsignale, wenn keine Hardware angeschlossen ist

Hardware

Mikrocontroller

Eigenschaft	Wert
Gerät	NXP Kinetis MK22F51212
Kern	ARM Cortex-M4 mit FPU
Kerntakt	120 MHz
Bustakt	60 MHz
Flash	256 KB
Zielboards	MC_CAR (100-polig), TinyK22 (64-polig)

Pin-Belegung

Schrittmotoren

Motor	Schritt-Pin	Richtungs-Pin	Enable-Pin
Motor 1 (Achse 1)	PA4	—	—
Motor 2 (Achse 2)	PA5	—	—
Motor 3 (Achse 3)	PD4	—	—
Rotationsmotor	PD5	—	—

Sensoren (Endschalter)

Sensor	Pin	Konfiguration
Sensor 1	PC8	Digitaleingang, Pull-up
Sensor 2	PC9	Digitaleingang, Pull-up
Sensor 3	PC10	Digitaleingang, Pull-up

Spule (Elektromagnet)

Signal	Pin
Spulensteuerung	PD0 (GPIO-Ausgang)

Debug- / Reserve-Pins

Signal	Pin
Debug 0	PA13

Signal	Pin
Debug 1	PA12
Debug 2	PD1
Debug 3	PD6

Timer — FTM0-Kanal-Zuordnung

FTM0 läuft mit 60 MHz Bustakt (konfigurierbarer Prescaler) und erzeugt Schritimpulse mit einer Pulsbreite von 100 µs.

Kanal	Funktion
CH1	Schritimpuls Motor 1
CH2	Schritimpuls Motor 2
CH4	Schritimpuls Motor 3
CH5	Schritimpuls Rotationsmotor
CH6	Rampen-Schritt-Inkrementierer (aktiv bei <code>RAMP_MODE_NSTEP</code>)

Kommunikation

UART-Schnittstelle

Der Mikrocontroller kommuniziert mit dem Raspberry Pi über **UART2** mit **115200 Baud**.

Befehle werden als **ReceivedCommand**-Struktur empfangen:

```
typedef struct {  
    int32_t Steps1;        // Schrittzahl Achse 1  
    int32_t Steps2;        // Schrittzahl Achse 2  
    int32_t Steps3;        // Schrittzahl Achse 3  
    int32_t StepsRot;      // Schrittzahl Rotationsmotor  
    bool    ActCoil;       // Elektromagnet aktivieren  
    bool    ErrorHandling; // Fehlerbehandlungsmodus aktivieren  
} ReceivedCommand;
```

Befehlssatz

Befehl	Beschreibung
start	Hauptbetriebsschleife starten
init	Alle Achsen über Endschalter in die Ausgangsposition fahren
camera	Kameraaufnahme auslösen
puzzleBegin	Puzzle-Lösungssequenz starten
puzzleEnd	Puzzle-Lösungssequenz beenden

Terminal-Handler

term.c implementiert einen Befehls-Dispatcher nach dem Registry-Muster. Jeder Befehl registriert eine Handler-Funktion. Beim Empfang eines vollständigen UART-Frames ruft der Dispatcher den passenden Handler auf.

Motorsteuerung

Schritimpuls-Erzeugung

Schritimpulse werden über **FTM0 Output-Compare-Interrupts** erzeugt. Jeder Motor hat einen dedizierten Kanal-ISR, der:

1. Den Schritt-Pin auf HIGH setzt
2. 100 µs wartet (Pulsbreite)
3. Den Schritt-Pin auf LOW setzt
4. Den nächsten Impuls basierend auf der konfigurierten Schrittpause plant

Die Schrittpause bestimmt die Motorgeschwindigkeit. Eine kürzere Pause bedeutet eine schnellere Bewegung.

Überlaufbehandlung

FTM0 verwendet einen 16-Bit-Zähler (max. 65535 Ticks). Bei großen Schrittzahlen verfolgt die ISR Überlaufzyklen über einen 32-Bit-Zähler, der dekrementiert wird, bis die verbleibenden Schritte in einen einzelnen Timer-Zyklus passen.

Rampenprofile

Drei Beschleunigungsrampen-Strategien stehen zur Verfügung (zur Kompilierzeit ausgewählt):

Modus	Beschreibung
<code>RAMP_MODE_PS</code>	Moduliert den FTM-Prescaler in 4 diskreten Stufen, um Anlaufstrom und Geräusche zu reduzieren
<code>RAMP_MODE_NSTEP</code>	Verwendet eine feste N-Schritt-Tabelle (Standard: 20 Schritte); FTM CH6 inkrementiert den Rampen-Index bei jedem Schritt
<code>RAMP_MODE_PREMIUM</code>	Berechnet eine lineare Rampe in der ISR bei jedem Schritt für die gleichmässigste Beschleunigung

Referenzfahrt

`moveToInitPos()` fährt alle Achsen langsam in Richtung der Endschalter. Wenn ein Sensor auslöst (aktiv LOW über Pull-up), stoppt die entsprechende Achse und ihre Position wird auf null gesetzt.

Erstellen & Flashen

Voraussetzungen

- MCUXpresso IDE (NXP) oder arm-none-eabi-gcc Toolchain
- J-Link oder OpenSDA Debug-Probe

Erstellen mit MCUXpresso

1. Projekt importieren: **File** → **Import** → **Existing Projects into Workspace**
2. Das Verzeichnis `PREN_Project_Share` auswählen
3. Die Build-Konfiguration **Debug** wählen
4. Auf **Build** klicken (oder `Ctrl+B` drücken)

Das Ausgabe-Artefakt ist `Debug/PREN_Project_Share.axf`.

Erstellen über die Kommandozeile

```
cd Debug  
make -j4
```

Flashen & Debuggen

Debug-Probe anschliessen und die integrierte **Debug**-Startkonfiguration von MCUXpresso verwenden, oder über den J-Link Commander flashen:

```
JLinkExe -device MK22FN512xxx12 -if SWD -speed 4000  
> loadfile Debug/PREN_Project_Share.axf  
> r  
> go
```

Debug-Modi

Folgende Flags in `globals.h` vor dem Erstellen setzen:

```
#define DEBUG_MODE      1    // Debug-Pin-Ausgabe aktivieren, Hauptschleife  
deaktivieren  
#define TEST_SEQUENCE  1    // Automatisierten Motortest beim Start ausführen  
#define ISR_MONITOR    1    // Reserve-Pins in ISRs für Timing-Analyse umschalten
```